

Agentic AI Applications Course

Team Number: _____

Team Members:

Aastha Gobind – R23EA002

Harshitha V – R23EA049

Rishi Tejas – R23EA098

Sriram Kundapur – R23EA122

Proposed Project Title:

ZeroGate: The Autonomous Security Graph

1. Problem Statement & Use Case

- **Monorepo Comprehension:** Developers and security teams lack an interactive, accurate way to query, understand, and map out codebase structures and relationships across large, multi-language monorepos.
- **Limitations of Keyword Search:** Standard text searching fails to find functions based on their actual purpose. There is a need for semantic, intent-based code search (e.g., searching for "error handling functions" instead of exact variable names).
- **Imprecise Code Modification:** AI code assistants often break syntax when modifying files. There is a need for surgical code replacement using Abstract Syntax Tree (AST)-based function targeting and visual diff previews.
- **Stale Context:** Codebases change rapidly. Systems analysing the code quickly become outdated unless they can watch for file changes and recalculate all function call relationships in real-time.
- **Fragmented Tooling:** Developers need a unified system that acts as a central knowledge graph, combining natural language querying, interactive AI-powered code optimization, and direct shell command execution.

2. Agentic Justification

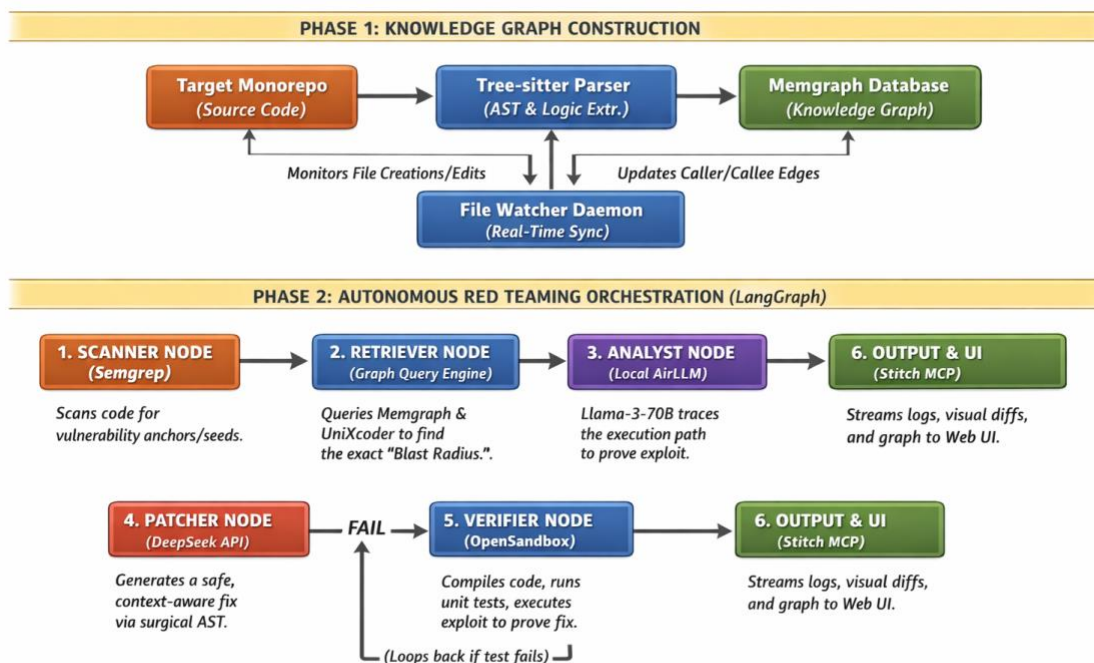
- **Natural Language to Graph Translation:** The system requires AI (specifically orchestrator and Cypher models) to translate plain English questions into precise database queries to navigate the codebase graph.
- **Semantic Intent Matching:** AI embeddings (specifically UNIX coder) are required to power intent-based code search, allowing the system to understand the meaning behind code blocks rather than just raw text.
- **Interactive Code Optimization:** AI is necessary to analyse the retrieved codebase context and provide intelligent optimization suggestions based on language-specific best practices.
- **Reference-Guided Operations:** The system uses AI to ingest custom coding standards and architectural documents, actively guiding the optimization and editing suggestions it makes.

- **Provider Flexibility & Cost Management:** An agentic routing setup is needed to mix different LLM providers (e.g., Ollama for local privacy/zero cost, vs. OpenAI/Google Gemini for high-accuracy cloud reasoning).

3. High-Level Solution Overview

- **Multi-language Parser:** A Tree-sitter-based parsing system actively analyses the codebase, extracting nested functions, class hierarchies, and dependencies, and ingests this data into a graph database.
- **Knowledge Graph Storage:** The system uses Memgraph to store the entire codebase structure as an interconnected graph, mapping exact caller/callee relationships.
- **Real-Time Synchronization:** A background watcher monitors the repository for file changes (create, modify, delete), filters out irrelevant files (like .git and node_modules), and automatically updates the knowledge graph in real-time.
- **Retrieval & Surgical Editing:** The RAG CLI retrieves actual source code snippets for found functions and enables surgical file editing with exact code block modifications and AST-based targeting.
- **MCP Server Integration:** ZeroGate Red Teaming functions natively as a Model Context Protocol (MCP) server, allowing AI assistants (like Claude Code) to seamlessly query and edit the codebase using natural language.

Block Diagram (Editable)



4. Proposed Technology Stack

- **Core Logic & CLI:** Python 3.12+ (managed via uv package manager).
- **Parsing Engine:** Tree-sitter (for multi-language AST parsing across C++, Java, JavaScript, Python, Lua, etc.).
- **Graph Database:** Memgraph (running via Docker & Docker Compose, utilizing cmake for pymgclient dependencies).
- **Embeddings & Search:** UniXcoder embeddings for semantic code search, combined with ripgrep (rg) for raw shell command text searching.
- **AI Inference Providers:** Support for Ollama (Local Models), OpenAI, and Google Gemini, configured via a provider-explicit .env setup.

5. Success Metrics & Evaluation Plan

- **Multi-Language Parsing Accuracy:** Successfully identifying and extracting complex language features such as C++ templates, C++20 modules, Java generics, and Python nested functions into the graph.
- **Real-Time Graph Consistency:** The ability of the repository watcher to accurately recalculate and update all function call relationships instantly when a file is modified.
- **Surgical Editing Precision:** High success rate in performing exact AST-based function replacements and block modifications without introducing structural syntax errors.
- **Semantic Search Relevance:** The accuracy of UniXcoder embeddings in successfully retrieving relevant code snippets based purely on natural language descriptions of their intent.
- **MCP Integration Reliability:** Seamless operation as an MCP server, successfully enabling external assistants like Claude Code to query the graph and edit files programmatically.

6. Risks, Challenges & Mitigation Plan

- **Risk:** Local LLMs (Ollama) providing lower accuracy compared to cloud models when generating graph queries.
 - **Mitigation:** Implement a mixed-provider configuration, allowing users to route complex orchestration tasks to Gemini/OpenAI while keeping simpler tasks local.
- **Risk:** Processing massive build directories and artifacts, which could crash the graph database.
 - **Mitigation:** Enforce strict file watchers that automatically filter out irrelevant directories like node_modules and .git.
- **Risk:** Breaking codebase structure during AI-driven file editing.
 - **Mitigation:** Mandate AST-based targeting and generate visual diff previews for interactive approval before applying any surgical code replacements.
- **Risk:** Failing to understand external dependencies outside the parsed source code.
 - **Mitigation:** Implement specific dependency analysis capabilities, such as parsing pyproject.toml to understand external project requirements.
- **Risk:** Missing complex, deeply nested codebase logic.
 - **Mitigation:** Utilize Tree-sitter's robust grammar to explicitly handle complex nested functions, class hierarchies, and multi-file module imports.

7. Timeline & Milestones

- **Milestone 1: Parser & Graph Foundation:** Implement the Python-based Tree-sitter parsing engine and establish the connection to ingest AST data into the Memgraph database.
- **Milestone 2: Real-Time Sync & CLI:** Build the interactive RAG system (codebase_rag/), implement the file watcher for real-time relationship recalculation, and integrate ripgrep.
- **Milestone 3: AI Provider Integration:** Integrate Ollama, OpenAI, and Google Gemini support, allowing natural language querying of the graph structure.
- **Milestone 4: Advanced Editing & Search:** Release semantic intent-based search utilizing UNIX coder embeddings, alongside surgical AST-based file editing and visual diffs.
- **Milestone 5: MCP & Ecosystem Integration:** Launch Model Context Protocol (MCP) server capabilities, enabling direct integration with AI agents like Claude Code, and add support for reference-guided optimization.